SOFTWARE DESIGN ANALYSIS REPORT

Department of Computer Science and Engineering | KIET Deemed to be University, Ghaziabad (AY: 2026-27)

A. PROJECT DETAILS

Project Title: **Multi-Vendor E-Commerce Platform (Version 1.0 Baseline)**

Team / Roll Numbers: Sparsh Tyagi (202401100200308), Vaishnavi Rajawat (202401100200335), Shubham Patel (202401100200299), Varnit Tomar (202401100200341)
Faculty Mentor: Mr. Rishabh Rathore Sir

Project Objective: To design, analyze, and build a self-contained, web-based e-commerce marketplace enabling multiple independent vendors to list and sell products to customers. The system provides complete end-to-end support for vendor onboarding, catalog moderation, multi-vendor shopping carts, order splitting, automated commission calculations, financial settlements, and administrative reporting.

Technology Stack: **Frontend Presentation:** Next.js / React with TypeScript, Tailwind CSS
Backend Core API: Node.js (LTS) REST APIs with Express & TypeScript
Data Layer & Cache: PostgreSQL (Relational DB), Prisma ORM, Redis (Session/Job Queues)
Integrations & DevOps: Razorpay/Stripe Payment Sandbox, Cloudinary (Object Storage), Docker Compose, GitHub Actions CI/CD

Architectural Sub-System Breakdown

The software is architected around a 3-tier decoupling paradigm to ensure complete separation of duties among presentation clients, central API services, and external integrations:

- **Client Storefront & Portals:** Dedicated responsive web interfaces designed for three distinct user roles: Customers (storefront browsing & checkout), Vendors (product catalog & sub-order management), and Administrators (system moderation, coupon rules & earnings settlement).
- **API & Business Logic Server:** Node.js backend executing stateless RESTful services, server-side Role-Based Access Control (RBAC), input sanitization, transactional inventory mutation, order splitting algorithms, and commission deduction logic.
- **Persistence & External Integration Layer:** ACID-compliant PostgreSQL database managed via Prisma ORM for schema consistency, Redis cache for session caching, Cloudinary object storage for product media, and Payment Gateway Webhook listeners for transactional verification.

B. MODULE IDENTIFICATION AND MODULARIZATION

Total Number of Modules: 6 Core Functional Modules
Modularity Rationale: The platform utilizes a modular decomposition strategy to split presentation routing, catalog processing, payment handling, and financial settlements into distinct logical blocks. This ensures high functional cohesion, minimal coupling, strict role boundaries, and allows team members to work on distinct subsystems concurrently without code conflicts.

Module ID	Module Name	Main Responsibility	Inputs / Outputs	Depends On
M1	Auth & Account Management	User registration, login, JWT token issuance, session control, password recovery, and server-side RBAC enforcement.	**In:** Credentials, Role **Out:** JWT, Auth Claims	Data Access Layer
M2	Vendor Onboarding & Profile	Manages seller applications, business KYC verification, admin approval/rejection workflows, and vendor storefront setup.	**In:** Business Info **Out:** Approval Status	M1
M3	Product Catalog & Search	Catalog listing management, variants/SKUs, image upload, category taxonomy structure, keyword search, and filtering.	**In:** Item Data, Queries **Out:** Listing Record	M2, Object Storage

Module ID	Module Name	Main Responsibility	Inputs / Outputs	Depends On
M4	Cart, Wishlist & Checkout	Multi-vendor cart aggregation, inventory stock check, coupon validation, subtotal calculation, and gateway trigger.	**In:** Cart Items, Coupon **Out:** Verified Order Intent	M3, Payment Gateway

B. Module Identification and Modularization (Continued)

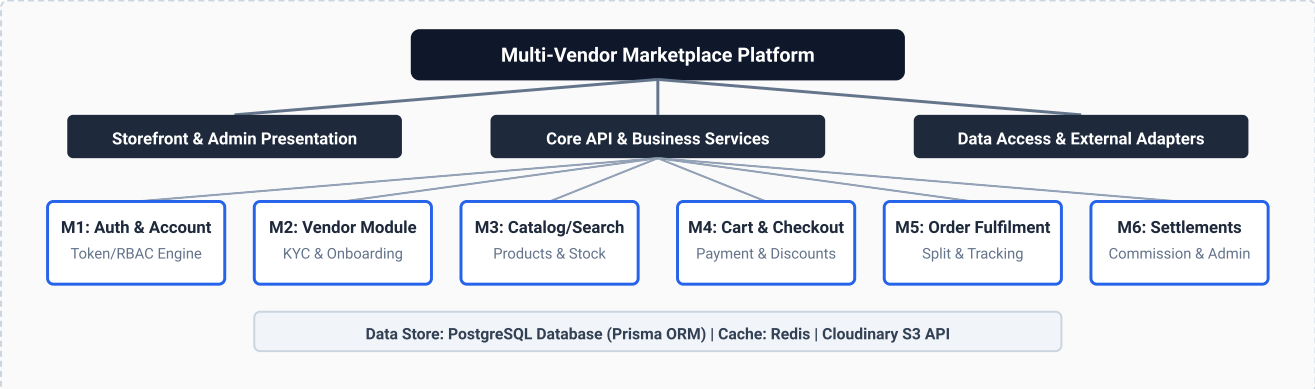
Module ID	Module Name	Main Responsibility	Inputs / Outputs	Depends On
M5	Multi-Vendor Order & Fulfilment	Splits multi-vendor orders into seller sub-orders, manages tracking state (Shipped, Delivered), returns, and inventory restoration.	**In:** Paid Intent, Status **Out:** Vendor Sub-Orders	M4, Shipping API
M6	Commission, Settlement & Admin	Computes platform commission rules, aggregates net vendor earnings, generates settlement statements, and feeds admin dashboards.	**In:** Order Totals, Rates **Out:** Payout Statements	M5

Detailed Sub-Module Breakdown

Each primary module is decomposed into focused sub-modules to guarantee granular task division:

- **M1 Sub-modules:** `M1.1 TokenAuthService` (JWT verification), `M1.2 UserProfileController` (Address/profile state), `M1.3 SecurityPolicyEnforcer` (RBAC & login throttling).
- **M2 Sub-modules:** `M2.1 ApplicationQueueManager` (Vendor signups), `M2.2 StorefrontCustomizer` (Banners/business info), `M2.3 StatusLifecycleEngine` (Admin approval/suspension).
- **M3 Sub-modules:** `M3.1 CatalogEditor` (Product variants/SKUs), `M3.2 TaxonomyManager` (Categories/attributes), `M3.3 SearchEngineAdapter` (Indexing, filtering & sorting).
- **M4 Sub-modules:** `M4.1 MultiVendorCartAggregator` (Session/User cart), `M4.2 CouponValidator` (Discount math), `M4.3 PaymentGatewayAdapter` (Razorpay/Stripe SDK integration).
- **M5 Sub-modules:** `M5.1 OrderSplitter` (Splits parent orders by vendor ID), `M5.2 FulfilmentTracker` (Shipping updates), `M5.3 ReturnRefundProcessor` (Returns workflow).
- **M6 Sub-modules:** `M6.1 LedgerCalculator` (Commission calculation engine), `M6.2 SettlementAggregator` (Weekly/Monthly vendor payouts), `M6.3 AdminAnalytics` (CSV exporter & metrics).

SYSTEM ARCHITECTURE & MODULE HIERARCHY DIAGRAM



C. DESIGN STRATEGY

☐ Top-Down ☐ Bottom-Up ☑ Hybrid (Top-Down & Bottom-Up)

Architectural Strategy Justification: The engineering team implemented a *Hybrid Design Strategy* to balance macro-level application partitioning with foundational component construction.

- **Top-Down Progression:** High-level system requirements from the SRS document were used to partition the marketplace into three primary web applications (Customer Storefront, Vendor Dashboard, Admin Dashboard) and core business API domains. This established overall dataflow boundaries, REST API end-points, and authorization scopes before writing feature logic.
- **Bottom-Up Construction:** Concurrently, foundational system abstractions were designed, tested, and implemented independently as reusable baseline modules. These included Prisma database schemas, BCrypt password hashing wrappers, JWT authentication middleware, global error-handling interceptors, and reusable Next.js UI component libraries (tables, status indicators, form inputs).
- **Integration Phase:** Top-down application routes were synthesized with bottom-up primitives to build complete end-to-end user workflows such as checkout order splitting and vendor commission calculation.

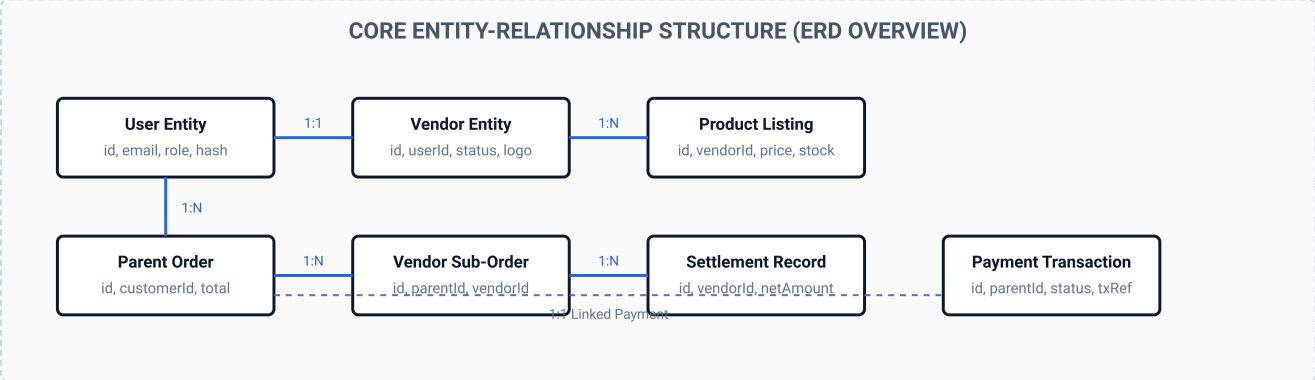
D. DESIGN APPROACH

☐ Function-Oriented ☑ Object-Oriented ☐ Mixed

Object-Oriented Paradigm Rationale: The project's problem domain natively aligns with Object-Oriented Analysis and Design (OOAD). Complex real-world entities—such as `User`, `VendorStore`, `ProductListing`, `ParentOrder`, `VendorSubOrder`, `PaymentTransaction`, and `SettlementStatement`—carry intrinsic state, identity, and strict state-transition rules.

- **Encapsulation:** Domain classes encapsulate business logic (e.g., `Order.calculateCommission()` or `Inventory.reserveStock()`), preventing external services from corrupting entity state directly.
- **Abstraction & Polymorphism:** Payment processing and email dispatches rely on polymorphic service interfaces (`IPaymentGateway`, `INotificationService`). This allows switching between Razorpay and Stripe sandboxes without altering core checkout logic.
- **Type Safety:** Built using TypeScript, ensuring strong typing across class constructors, interface payloads, and ORM database mappings.

Domain Entity Relationship & Data Model Structure



E. COUPLING ANALYSIS

Coupling measures the degree of interdependence between software modules. Low coupling is prioritized to ensure changes in one module do not ripple across the platform.

Module Pair	Interaction / Shared Data	Coupling Type / Level	Strategy to Reduce Coupling
M4 (Checkout) → M3 (Catalog)	Checkout service validates stock counts, prices, and variants against the product catalog prior to payment.	**Data Coupling** (Low / Desirable)	Pass minimal primitive parameters (`productId`, `requestedQty`) rather than whole domain objects. Cache catalog price reads in Redis.
M4 (Checkout) → M5 (Fulfilment)	Payment completion event triggers M5 to split parent order into vendor-specific sub-orders.	**Control / Stamp Coupling** (Medium-Low)	Decouple via an asynchronous message event bus (`PAYMENT_AUTHORIZED_EVENT`), replacing direct inter-service invocation.
M5 (Fulfilment) → M6 (Settlement)	Order delivery confirmation transfers completed sub-order financial totals into vendor settlement ledgers.	**Data Coupling** (Low)	Write immutable ledger line items (`VendorEarningsRecord`) upon state transitions rather than direct database table sharing.
M1 (Auth) → All Modules	Authentication middleware verifies user identity and attaches role permissions to HTTP request contexts.	**Stamp Coupling** (Low-Medium)	Attach a standardized, read-only `UserContext` payload (ID, Role, TenantID) to request headers via decoupled gateway middleware.

F. COHESION ANALYSIS

Cohesion evaluates how strongly focused the internal responsibilities of a module are. High functional cohesion is enforced across all sub-systems.

Module	Main Responsibility	Cohesion Type / Level	Design Refinement Applied
M1 (Auth & RBAC)	Manages user identities, credential validation, token signing, and role permission enforcement.	**Functional Cohesion** (High / Optimal)	Strictly limited to access control logic. User profile modification and address storage were delegated to `M1.2 UserProfileController`.
M4 (Checkout & Cart)	Handles multi-vendor cart calculation, stock verification, coupon evaluation, and payment initiation.	**Sequential Cohesion** (High)	Organized into sequential pipeline steps: `ValidateCart` → `ApplyCoupon` → `LockStock` → `InitiatePayment`.
M5 (Order Fulfilment)	Processes multi-vendor order splitting, vendor order queues, shipment tracking, and return processing.	**Communicational Cohesion** (High)	Operates on shared order entities. Third-party shipping carrier APIs were isolated into a dedicated gateway driver.
M6 (Settlements)	Calculates platform commissions, aggregates net vendor payouts, and outputs financial statements.	**Functional Cohesion** (High / Optimal)	Dedicated exclusively to financial math and reporting ledgers. Devoid of any presentation or user routing logic.

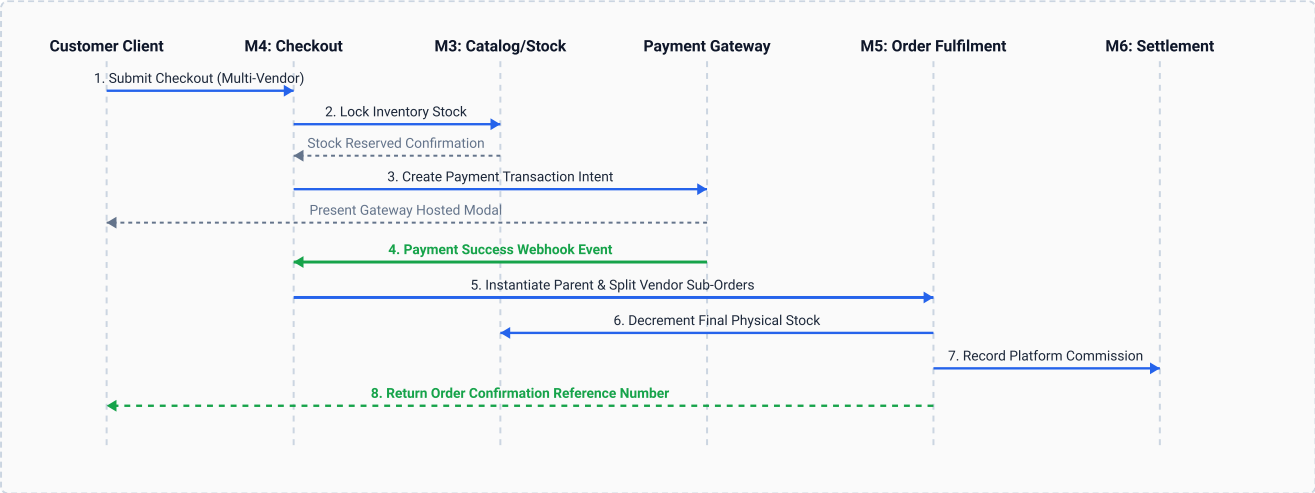
Data Integrity & Safety Controls

To comply with non-functional safety requirements (NFR-005, NFR-006, BR-04, BR-15):

- **Zero Negative Inventory:** Stock decrement commands are wrapped inside ACID-compliant PostgreSQL database transactions with strict row-level locking (`SELECT ... FOR UPDATE`) to prevent race conditions during concurrent checkouts.
- **Idempotent Payments:** Webhook processing engines verify payment signature hashes and check transaction idempotency keys in Redis to guarantee duplicate payment retries never cause double-order creation.
- **Capped Refund Protection:** Refund processing controllers validate that total cumulative refunds on any vendor order never exceed the initial captured payment amount.

G. MODULE INTERACTION AND DESIGN QUALITY

Sequence Diagram — Multi-Vendor Order Placement & Splitting Workflow:
The sequence diagram below models the dynamic interaction and data exchange across modules during checkout, inventory reservation, payment gateway webhook verification, order splitting, and vendor settlement tracking.



Design Quality & Maintainability Metrics

The software architecture achieves high operational readiness through the following design practices:

- **Testability (NFR-030):** Core business rule handlers (`OrderSplitter`, `CommissionCalculator`) are pure functions and class services isolated from UI components, allowing automated unit testing via Jest without mocking database connections.
- **Extensibility & Maintainability (NFR-025):** Adding new system features—such as multi-currency conversion or native mobile app push notifications—requires adding discrete service adapters without modifying existing core data models.
- **Auditing & Traceability (NFR-014):** Critical state transitions (vendor approvals, status suspensions, order cancellations, and refund approvals) generate immutable audit logs recording timestamp, acting user ID, IP address, and changed attributes.